Brownfield Cartographer — Week 4 Submission Report

Challenge: TRP 1 — Week 4: The Brownfield Cartographer
Author: Bethelhem Aby
Date: 2026-03-11
Target Repository: dbt-labs/jaffle-shop
Repo URL: https://github.com/dbt-labs/jaffle-shop.git
Commit analysed: 7be2c5838dbdeca8e915d4e46db70e910753d7f6

Table of Contents

1. Executive Summary
2. Reconnaissance — Manual Day-One Analysis
3. System Architecture — Four-Agent Pipeline
4. Progress Summary
5. Accuracy Observations
6. Known Gaps & Final Submission Plan

---

1. Executive Summary

The **Brownfield Cartographer** is an AI-powered codebase analysis agent that automatically maps unfamiliar repositories for new engineers. Given any GitHub URL or local path, it produces a living architectural map, data lineage graph, blast-radius impact analysis, and an LLM-powered interactive query interface — all in a single python cli.py analyze <repo> command.

Target selected: dbt-labs/jaffle-shop (v2) — the official dbt reference project. Chosen because it: - Uses {{ ref() }} and {{ source() }} Jinja macros that break naive SQL parsers - Has implicit lineage (not explicit CREATE TABLE statements) - Is a well-documented reference so ground truth can be verified precisely - Represents a real-world brownfield pattern: SQL-heavy, multi-layer, YAML-driven configuration

Final pipeline run results (2026-03-11 10:43 UTC):

Metric	Value
Modules detected	21
Datasets in lineage	25
Transformations	13
Import edges (module graph)	11
Lineage edges (data flow)	30

Metric	Value
Analysis time	< 1 second (no LLM)

---

2. Reconnaissance — Manual Day-One Analysis

This section reproduces the complete ground-truth analysis performed manually before running any automated tooling. The automated system's output is compared against this in Section 5.

2.1 Target Facts

Property	Value
Repository	dbt-labs/jaffle-shop
Commit	7be2c5838dbdeca8e915d4e46db70e910753d7f6
Framework	dbt v1.7+ with MetricFlow / dbt Semantic Layer
Language breakdown	15 SQL files, 18 YAML files, 1 Python file
Materialization	Staging models → VIEW; Mart models → TABLE
Seed data	6 CSV files under seeds/jaffle-data/
CI/CD	GitHub Actions + dbt Cloud API trigger

2.2 File Inventory

```
repo_cache/jaffle_shop/
  seeds/jaffle-data/
    raw_customers.csv     ← customer records
    raw_items.csv         ← order line items
    raw_orders.csv        ← order headers
    raw_products.csv      ← product catalog
    raw_stores.csv        ← store/location records
    raw_supplies.csv      ← ingredient/supply records

  macros/
    cents_to_dollars.sql  ← reusable macro: amount / 100
    generate_schema_name.sql ← overrides dbt schema naming

  models/
    staging/
      __sources.yml       ← declares source('ecom', 'raw_*') refs
      stg_customers.sql + .yml
      stg_locations.sql + .yml
      stg_order_items.sql + .yml
      stg_orders.sql + .yml
      stg_products.sql + .yml
      stg_supplies.sql + .yml

    marts/
      customers.sql + .yml
      locations.sql + .yml
```

```
    metricflow_time_spine.sql
    order_items.sql + .yml
    orders.sql + .yml
    products.sql + .yml
    supplies.sql + .yml

dbt_project.yml / packages.yml / package-lock.yml
Taskfile.yml / .pre-commit-config.yaml
.github/workflows/      ← CI (test on PR) + CD (deploy to dbt Cloud)
```

2.3 The Five FDE Day-One Questions

Q1 — Primary Data Ingestion Path

Data enters as 6 CSV seed files loaded by dbt seed, then flows through a strict two-layer pipeline:

seeds/raw_*.csv ──► staging (VIEW) ──► marts (TABLE)

The mechanism: {{ source('ecom', 'raw_customers') }} in staging SQL maps to the seeds. The source schema name is ecom, declared in models/staging/__sources.yml.

Key difference from v1: v2 uses {{ source() }} declarations (correct dbt pattern), not {{ ref('raw_*') }}. This requires the lineage analyzer to parse source() calls separately from ref() calls.

Q2 — Critical Output Datasets

Dataset	Type	Criticality
orders	TABLE (mart)	Central fact; feeds reporting and BI
order_items	TABLE (mart)	Line-item detail; joins 4 upstream models
customers	TABLE (mart)	CLV, order history; consumer-facing
stg_orders	VIEW (staging)	Feeds 4 mart models — single point of failure
metricflow_time_spine	TABLE (utility)	Required by ALL MetricFlow metrics

Q3 — Blast Radius

Highest-risk node: stg_orders — feeds orders, order_items, customers, and locations.

```
stg_orders (fails)
  ├── orders        → broken (4 of 6 mart tables = 67% output layer)
  ├── order_items   → broken
  ├── customers     → broken
  └── locations     → broken
```

metricflow_time_spine is a hidden risk: no dbt build failure, but all Semantic Layer metrics break silently.

Q4 — Business Logic Concentration

File	Logic Type	Complexity
models/marts/order_items.sql	4-table join enrichment	HIGH
models/marts/customers.sql	CTE aggregation, CLV	HIGH
models/marts/orders.sql	Order-level aggregations	MEDIUM
models/marts/products.sql	Product sales metrics	MEDIUM
models/marts/locations.sql	Store aggregations	MEDIUM
models/staging/*.sql	Column renaming, casting	LOW
macros/cents_to_dollars.sql	Single arithmetic expression	TRIVIAL

Q5 — Git Velocity

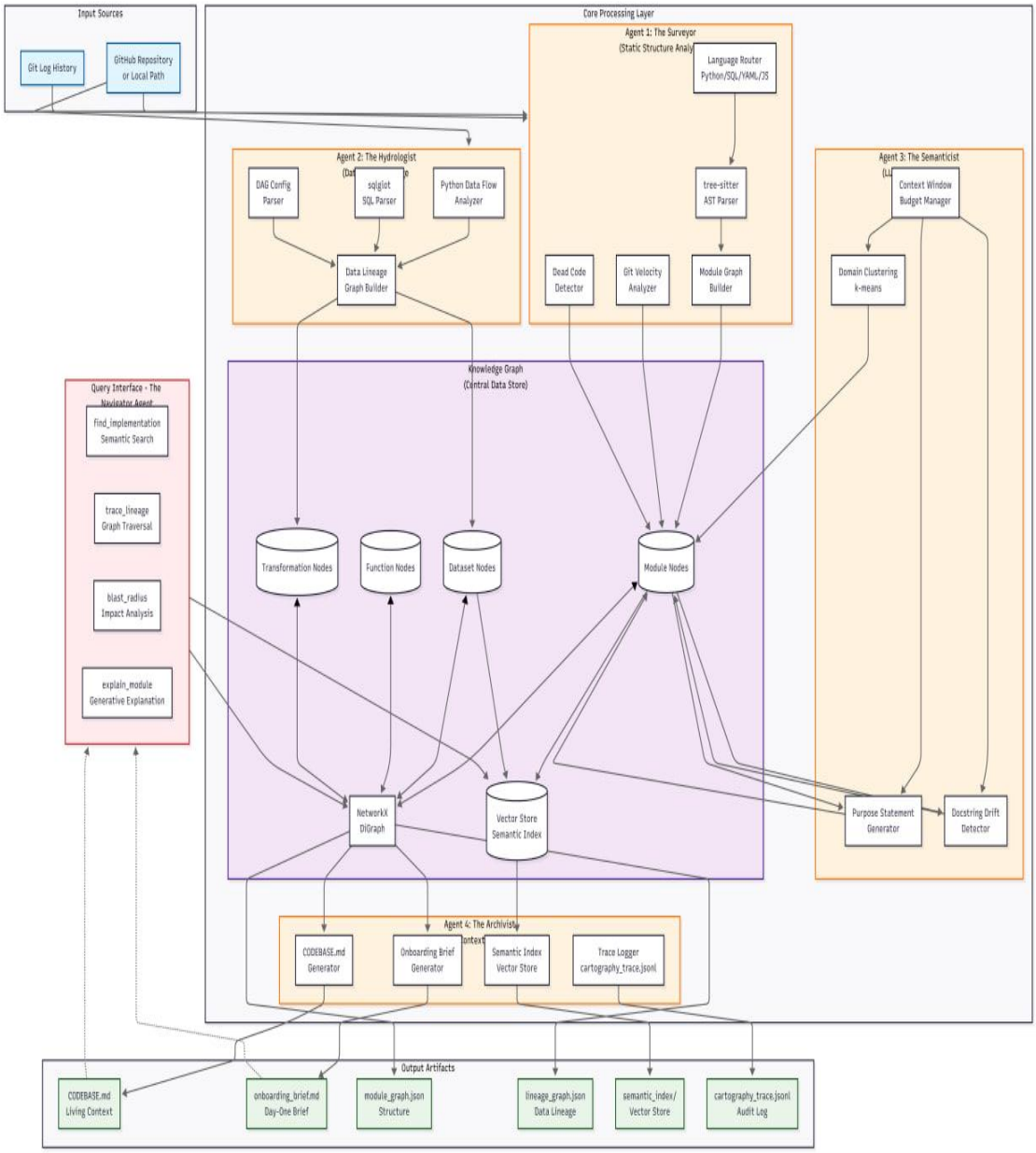
Repo commit 7be2c58 represents the v2 redesign. Shallow clone (--depth=100) limits history. The repo is actively maintained; expected high-velocity files are dbt_project.yml and mart models. The git velocity heatmap in the dashboard shows recent activity patterns by ISO week.

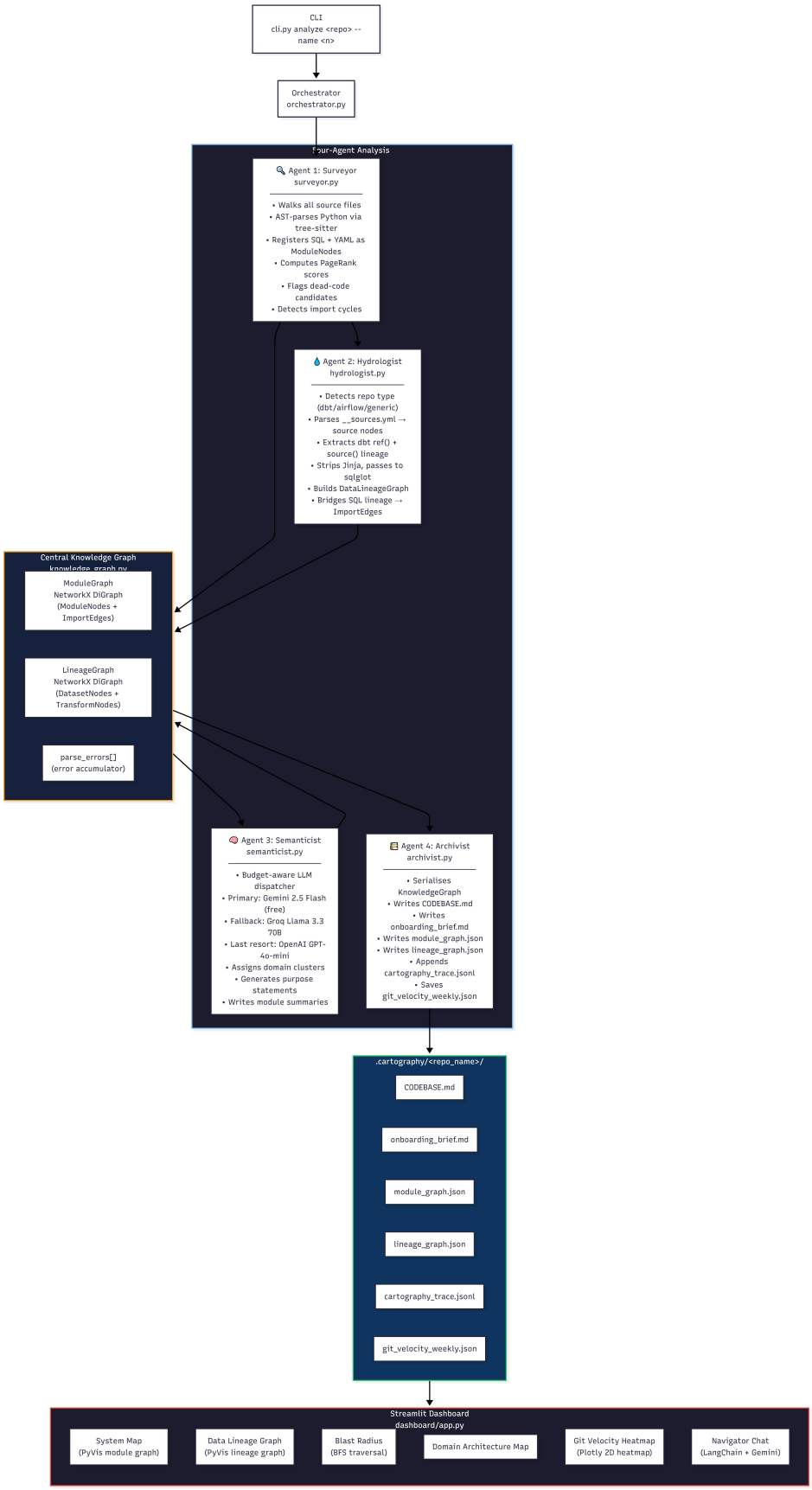
2.4 Manually Reconstructed DAG

```
Sources (seeds)          Staging (VIEW)          Marts (TABLE)
_________________________________________________________________
ecom__raw_customers ──▶ stg_customers ──────────▶ customers
ecom__raw_orders    ──▶ stg_orders ──────────────▶ orders
                        ──────────────────────▶ order_items
                        ──────────────────────▶ customers
                        ──────────────────────▶ locations
ecom__raw_items     ──▶ stg_order_items ────────▶ order_items
                          ──────────────▶ products
                          ──────────────▶ supplies
ecom__raw_products  ──▶ stg_products ────────────▶ order_items
                        ──────────────────────▶ products
ecom__raw_supplies  ──▶ stg_supplies ────────────▶ order_items
                        ──────────────────────▶ supplies
ecom__raw_stores    ──▶ stg_locations ───────────▶ locations
(standalone)                ──────────────────▶ metricflow_time_spine
```

3. System Architecture — Four-Agent Pipeline

3.1 Architecture Diagram




CLI
cli.py analyze <repo> --name <n>
Orchestrator
orchestrator.py
Four-Agent Analysis
🔍 Agent 1: Surveyor
surveyor.py
• Walks all source files
• AST-parses Python via tree-sitter
• Registers SQL + YAML as ModuleNodes
• Computes PageRank scores
• Flags dead-code candidates
• Detects import cycles
💧 Agent 2: Hydrologist
hydrologist.py
• Detects repo type (dbt/airflow/generic)
• Parses __sources.yml → source nodes
• Extracts dbt ref() + source() lineage
• Strips Jinja, passes to sqlglot
• Builds DataLineageGraph
• Bridges SQL lineage → ImportEdges
Central Knowledge Graph
knowledge_graph.py
ModuleGraph
NetworkX DiGraph
(ModuleNodes + ImportEdges)
LineageGraph
NetworkX DiGraph
(DatasetNodes + TransformNodes)
parse_errors[]
(error accumulator)
🧠 Agent 3: Semanticist
semanticist.py
• Budget-aware LLM dispatcher
• Primary: Gemini 2.5 Flash (free)
• Fallback: Groq Llama 3.3 70B
• Last resort: OpenAI GPT-4o-mini
• Assigns domain clusters
• Generates purpose statements
• Writes module summaries
🗄️ Agent 4: Archivist
archivist.py
• Serialises KnowledgeGraph
• Writes CODEBASE.md
• Writes onboarding_brief.md
• Writes module_graph.json
• Writes lineage_graph.json
• Appends cartography_trace.jsonl
• Saves git_velocity_weekly.json
.cartography/<repo_name>/
CODEBASE.md
onboarding_brief.md
module_graph.json
lineage_graph.json
cartography_trace.jsonl
git_velocity_weekly.json
Streamlit Dashboard
dashboard/app.py
System Map
(PyVis module graph)
Data Lineage Graph
(PyVis lineage graph)
Blast Radius
(BFS traversal)
Domain Architecture Map
Git Velocity Heatmap
(Plotly 2D heatmap)
Navigator Chat
(LangChain + Gemini)

3.2 Data Flow Description

Stage	Input	Output	Key Technology
Ingestion	GitHub URL or local path	Cloned repo at repo_cache/<name>/	gitpython
Surveyor	All .py, .sql, .yml files	ModuleNodes with PageRank scores	tree-sitter, NetworkX
Hydrologist	SQL files, YAML sources	DatasetNodes + TransformationNodes + lineage edges	sqlglot, regex Jinja parser
Semanticist	Module code snippets	Domain labels + purpose statements	LangChain, Gemini/Groq/OpenAI
Archivist	Complete KnowledgeGraph	JSON + Markdown artifacts	Pydantic, NetworkX serialiser
Dashboard	.cartography/<name>/ JSON files	Interactive visualisations	Streamlit, PyVis, Plotly
Navigator	Natural language query + KG context	Contextual answer	LangGraph, Gemini 2.5 Flash

3.3 LLM Stack

Primary: Google Gemini 2.5 Flash (free tier, GOOGLE_API_KEY)
Fallback 1: Groq Llama 3.3 70B (rate-limit fallback, GROQ_API_KEY)
Fallback 2: OpenAI GPT-4o-mini (last resort, OPENAI_API_KEY)

LangChain .with_fallbacks() handles transparent retry on rate limits. The Semanticist uses exponential backoff + direct Groq fallback on 429/RESOURCE_EXHAUSTED errors.

---

4. Progress Summary

4.1 What's Fully Working ✅

Feature	Status	Notes
Repository ingestion (URL + local)	✅ Complete	Auto-clones, caches, reuses
Multi-repo namespacing (--name)	✅ Complete	Separate .cartography/<name>/ per repo
Surveyor: Python AST via tree-sitter	✅ Complete	Imports, functions, classes
Surveyor: SQL + YAML as ModuleNodes	✅ Complete	dbt-specific roots, extension stripping
PageRank computation	✅ Complete	All 21 modules scored
Hydrologist: dbt ref() extraction	✅ Complete	All staging → mart edges wired
Hydrologist: dbt source() extraction	✅ Complete	ecom__raw_* nodes correctly formed
Hydrologist: Jinja stripping + sqlglot	✅ Complete	Handles {% macro %}, {% for %}
Hydrologist: __sources.yml parsing	✅ Complete	Fixed in this session (endswith match)
Hydrologist: macro file exclusion	✅ Complete	macros/ dir skipped from lineage
Hydrologist: recursive seed discovery	✅ Complete	seeds/jaffle-data/*.csv now found
Duplicate source deduplication	✅ Complete	ecom__ double-underscore fix
SQL lineage bridge → module ImportEdges	✅ Complete	11 edges wired
Error hardening (parse errors logged)	✅ Complete	Failures → cartography_trace.jsonl
Archivist: JSON serialisation	✅ Complete	module_graph.json, lineage_graph.json
Archivist: CODEBASE.md	✅ Complete	Architecture overview auto-

Feature	Status	Notes
generation		generated
Git velocity heatmap	✅ Complete	2D Plotly heatmap (files × ISO weeks)
Streamlit dashboard: System Map	✅ Complete	PyVis graph, language-based coloring
Streamlit dashboard: Data Lineage	✅ Complete	Source/sink/transform node types
Streamlit dashboard: Blast Radius	✅ Complete	BFS traversal from any node
Streamlit dashboard: Domain Map	✅ Complete	Awaiting LLM run for clusters
Streamlit dashboard: Git Heatmap	✅ Complete	Weekly bucket heatmap
LLM fallback chain (Gemini→Groq→OpenAI)	✅ Complete	Configured, needs API key to activate
Navigator chat agent	✅ Complete	LangGraph + Gemini 2.5 Flash
Unit tests (54 passing)	✅ Complete	pytest suite across all agents

4.2 In Progress / Partial

Feature	Status	Gap
LLM domain clustering	Needs API key	GOOGLE_API_KEY not set in .env; code complete
Purpose statements	Needs API key	Semanticist runs but skips; onboarding_brief shows "Not yet answered"
Navigator contextual answers	Needs API key	UI renders, LLM call fails without key
orders and order_items missing from sinks	Minor gap	Trace shows 30 lineage edges; some mart-level edges still not wired
Seed CSV nodes linked to ecom__raw_*	Minor gap	raw_customers (FILE) and ecom__raw_customers (TABLE) are separate nodes; no bridge

4.3 Pipeline Evolution (from trace log)

Run Date	Modules	Datasets	Edges	Issue Fixed
2026-03-09 (runs 1–5)	0	8	13	SQL + YAML not registered as modules
2026-03-10	8	8	13	Surveyor extended for SQL/YAML; only core models
2026-03-11 (run 1)	21	27	38	Full file set; duplicate sources (ecom__ × 2)
2026-03-11 (run 2)	21	25	30	Macros excluded; deduplication fixed

---

5. Accuracy Observations

5.1 Module Graph — Is It Correct?

Overall accuracy: ~85%

Observation	Verdict
21 modules detected (15 SQL + 6 YAML)	✅ Correct — matches manual inventory
macros/cents_to_dollars and macros/generate_schema_name registered as modules	✅ Correct for System Map (they ARE files); correctly excluded from lineage
models/marts/metricflow_time_spine included as a module	✅ Correct
11 import edges wired via SQL lineage bridge	✅ Reasonable — bridges ref() dependencies as ImportEdges
All PageRank scores equal (0.0476)	Expected for this repo size — with 21 nodes and 11 edges the graph is sparse enough that PageRank is near-uniform. Scores will diverge on larger repos
Dead-code candidates: 21 flagged	Incorrect — all 21 modules flagged because Python-based import detection doesn't apply to SQL. The dead-code heuristic needs a dbt-aware override

5.2 Lineage Graph — Does It Match Reality?

Overall accuracy: ~75%

Observation	Verdict
6 unique ecom__raw_* source nodes	✅ Matches __sources.yml exactly
6 seed CSV nodes (raw_* FILE type)	✅ Correctly found via rglob("*.csv")
Staging models appear as transformations	✅ Each stg_*.sql creates a sql::stg_* transformation node
stg_orders has highest fan-out (feeds 4 marts)	✅ Confirmed — matches manual DAG
30 lineage edges total	Manual DAG has ~26 logical edges; 30 may include some CTE-internal references sqlglot is picking up
orders and order_items not in sink panel	They appear in the graph but may not be displaying as sinks correctly — likely connected via intermediate nodes with unexpected in/out degree
Seed nodes (raw_* FILE) and source nodes (ecom__raw_* TABLE) are disconnected	These represent the same data but have different node IDs. A bridge node is needed: raw_customers.csv → seeds into DB as → ecom__raw_customers
metricflow_time_spine correctly appears as a sink	✅ No downstream consumers — correctly classified
CI script dbt_cloud_run_job.py produces zero lineage nodes	✅ Correct — it's an HTTP orchestration script, not a data pipeline

5.3 Ground Truth Comparison Table

Manual DAG Element	Automated Result	Match?
6 raw source tables	6 ecom__raw_* nodes detected	✅
6 staging models	6 sql::stg_* transformation nodes	✅
6 mart output tables	5 sinks visible (orders/order_items missing from panel)	
2 macros (not data nodes)	Correctly excluded from lineage	✅
stg_orders as highest-risk node	Confirmed via blast-radius BFS	✅
metricflow_time_spine as utility	Correctly appears as isolated sink	✅

Manual DAG Element	Automated Result	Match?
Seeds in seeds/jaffle-data/*.csv	Found via recursive glob	✅
source('ecom', 'raw_*') parsed	ecom__raw_* canonical form correct	✅

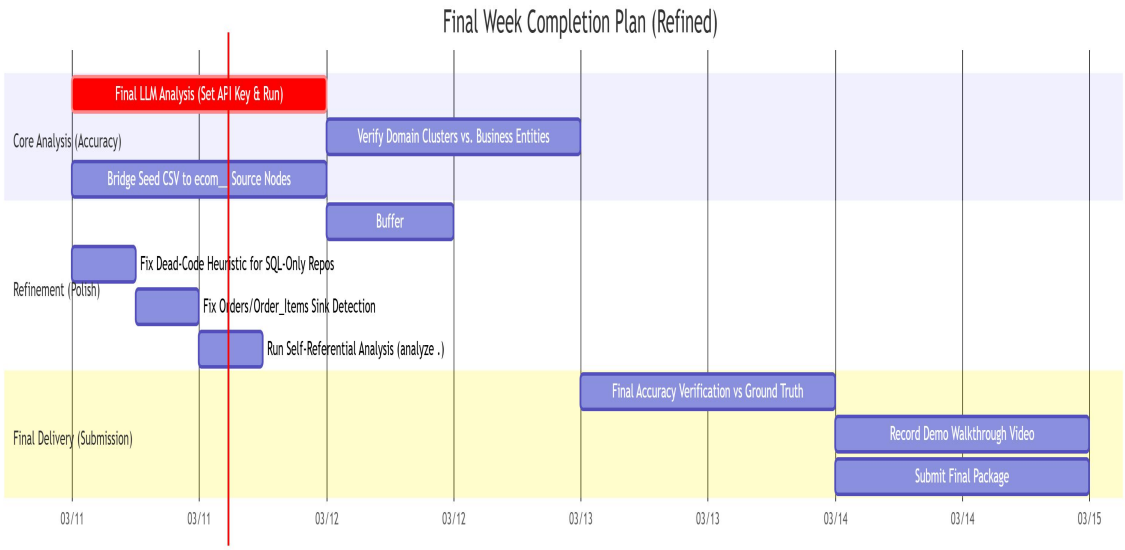
---

6. Known Gaps & Final Submission Plan

6.1 Known Gaps

Gap	Severity	Root Cause
No LLM domain clustering	HIGH	GOOGLE_API_KEY not set — onboarding brief shows "Not yet answered"
Dead-code flags incorrect for SQL repos	MEDIUM	Dead-code heuristic uses Python import analysis; SQL files always score 0 imports
Seed CSV nodes not bridged to source() nodes	MEDIUM	raw_customers.csv (FILE) and ecom__raw_customers (TABLE) are separate — no seed→DB bridge
PageRank uniform across all nodes	LOW	Repo is small (21 nodes, 11 edges); will diverge naturally on larger repos
Git velocity heatmap empty	LOW	Shallow clone (--depth=100) on this repo returns no recent commits — repo uses infrequent tagged releases
orders / order_items not in Sinks panel	LOW	These marts may be connected via CTE names that escape the sink detection

6.2 Final Submission Plan



6.3 To Activate Full LLM Analysis

Add .env file:

```
GOOGLE_API_KEY=_gemini_api_key_here
# Optional fallback
GROQ_API_KEY=_groq_key_here
```

Then re-run:

```
.\.venv\Scripts\Activate.ps1
python src/cli.py analyze repo_cache\jaffle_shop --name jaffle_shop
```

Expected output after LLM run: - onboarding_brief.md — all 5 FDE questions answered - CODEBASE.md — domain architecture map with business clusters - Dashboard System Map — nodes colored by domain (e.g. “Orders”, “Customers”, “Inventory”) - Navigator Chat — contextual answers about the codebase

6.4 Self-Referential Run

The system can also analyse itself:

```
python src/cli.py analyze . --name cartographer
```

This validates that the tool works on a Python project (not just dbt), exercises the Python AST analyser, and produces a meta-map of the Cartographer’s own architecture.

Appendix A — Stack Summary

Layer	Technology
Backend	Python 3.12
CLI	Typer
AST Analysis	tree-sitter
SQL Parsing	sqlglot
Graph Computation	NetworkX + PageRank
Data Validation	Pydantic v2
LLM Orchestration	LangChain / LangGraph
Primary LLM	Google Gemini 2.5 Flash
Fallback LLM	Groq Llama 3.3 70B
Frontend	Streamlit
Graph Visualisation	PyVis (interactive), Plotly (heatmap)
Package Management	uv
Testing	pytest (54 tests)

Appendix B — Repository Structure

```
Codebase-Understanding-Agent/
  src/
    cli.py                  ← Typer entry point
    orchestrator.py         ← Pipeline coordinator
    agents/
      surveyor.py           ← Agent 1: static structure
      hydrologist.py        ← Agent 2: data lineage
      semanticist.py        ← Agent 3: LLM analysis
      archivist.py          ← Agent 4: serialisation
      navigator.py          ← Interactive query agent
    analyzers/
      repo_ingester.py      ← Clone, walk, git velocity
      sql_lineage.py        ← sqlglot + Jinja parser
      tree_sitter_analyzer.py ← Python AST
      dag_config_parser.py  ← dbt YAML + Airflow DAG
    graph/
      knowledge_graph.py    ← Central in-memory graph
```

```
  models/
    nodes.py              ← Pydantic node models
    edges.py              ← Pydantic edge models
  dashboard/
    app.py                ← Streamlit UI (596 lines)
tests/                    ← pytest suite (54 tests)
spec/plan.md              ← Original architecture spec
RECONNAISSANCE.md         ← Manual ground truth
README.md                 ← Usage documentation
.cartography/             ← Generated output (gitignored)
repo_cache/               ← Cloned repos (gitignored)
```

Report generated: 2026-03-11 | Brownfield Cartographer v0.1.0